

Отчет по домашнему заданию. Блок №1

Метод кубической аппроксимации (интерполяция Эрмита)

1 Постановка задачи

Согласно варианту 19, задана функция:

$$f(x) = \frac{1}{2}x^2 - \sin(x), \quad x \in [0, 1] \quad (1)$$

Требуется:

1. Реализовать поиск минимума методом кубической аппроксимации с точностью $\varepsilon = 0.0001$.
2. Построить визуализацию исходной функции $f(x)$ и аппроксимирующего многочлена $H(x)$ для **каждой** итерации алгоритма.

2 Математическое описание метода

Метод базируется на построении кубического многочлена Эрмита $H(x)$ на текущем отрезке $[x_0, x_1]$. Используются значения функции y_0, y_1 и её производных y'_0, y'_1 .

Коэффициенты многочлена $H(x) = a_3x^3 + a_2x^2 + a_1x + a_0$ вычисляются по формулам из `kubinter_2.pdf`:

- $\Delta = x_1 - x_0$
- $A = y_0, \quad B = y'_0 + \frac{2y_0}{\Delta}, \quad C = y_1, \quad D = -y'_1 + \frac{2y_1}{\Delta}$
- $a_3 = B - D$
- $a_2 = (A - Bx_0 - 2Bx_1) + (C + Dx_1 + 2Dx_0)$
- $a_1 = (-2A + 2Bx_0 + Bx_1)x_1 + (-2C - 2Dx_1 - Dx_0)x_0$
- $a_0 = (A - Bx_0)x_1^2 + (C + Dx_1)x_0^2$

Точка минимума x_m находится из условия $H'(x) = 0$:

$$x_m = \frac{-2a_2 + \sqrt{4a_2^2 - 12a_3a_1}}{6a_3} \quad (2)$$

3 Программная реализация и результаты

Ниже представлен код на Python, выполняющий расчет и генерацию графиков для всех итераций.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f(x): return 0.5 * x**2 - np.sin(x)
5 def df(x): return x - np.cos(x)
6
7 def solve():
```

```

8     a, b = 0.0, 1.0
9     eps = 0.0001
10    history = []
11
12    for iter_num in range(1, 11):
13        x0, x1 = a, b
14        y0, y1, d0, d1 = f(x0), f(x1), df(x0), df(x1)
15        delta = x1 - x0
16
17        #                                     A, B, C, D
18        A, B = y0, d0 + 2*y0/delta
19        C, D = y1, -d1 + 2*y1/delta
20
21        #
22        a3 = B - D
23        a2 = (A - B*x0 - 2*B*x1) + (C + D*x1 + 2*D*x0)
24        a1 = (-2*A + 2*B*x0 + B*x1)*x1 + (-2*C - 2*D*x1 - D*x0)*x0
25        a0 = (A - B*x0)*x1**2 + (C + D*x1)*x0**2
26
27        #                                     H'(x)
28        discr = (2*a2)**2 - 4*(3*a3)*a1
29        xm = (-2*a2 + np.sqrt(discr)) / (6*a3)
30
31        history.append({'a':a, 'b':b, 'xm':xm, 'coeffs':(a3, a2, a1, a0
32    )})
33
34        print(f"Iter {iter_num}: xm = {xm:.6f}, f'(xm) = {df(xm):.6e}")
35
36        if abs(df(xm)) < eps:
37            break
38
39        if df(xm) < 0: a = xm
40        else: b = xm
41
42    return history
43
44    history = solve()
45
46    #
47    for i, step in enumerate(history):
48        plt.figure(figsize=(8, 5))
49        x_plot = np.linspace(0, 1, 100)
50        plt.plot(x_plot, f(x_plot), 'k--', label='f(x)')
51
52        a3, a2, a1, a0 = step['coeffs']
53        h_x = a3*x_plot**3 + a2*x_plot**2 + a1*x_plot + a0
54        plt.plot(x_plot, h_x, 'r', label=f'H(x) {i+1}')
55
56        plt.scatter(step['xm'], f(step['xm']), color='blue')
57        plt.axvspan(step['a'], step['b'], color='yellow', alpha=0.2, label=
58    ,
59    )
60        plt.title(f" {i+1}: x_m = {step['xm']:.5f}")
61        plt.legend()

```

```
59 plt.grid(True)
60 plt.show()
```

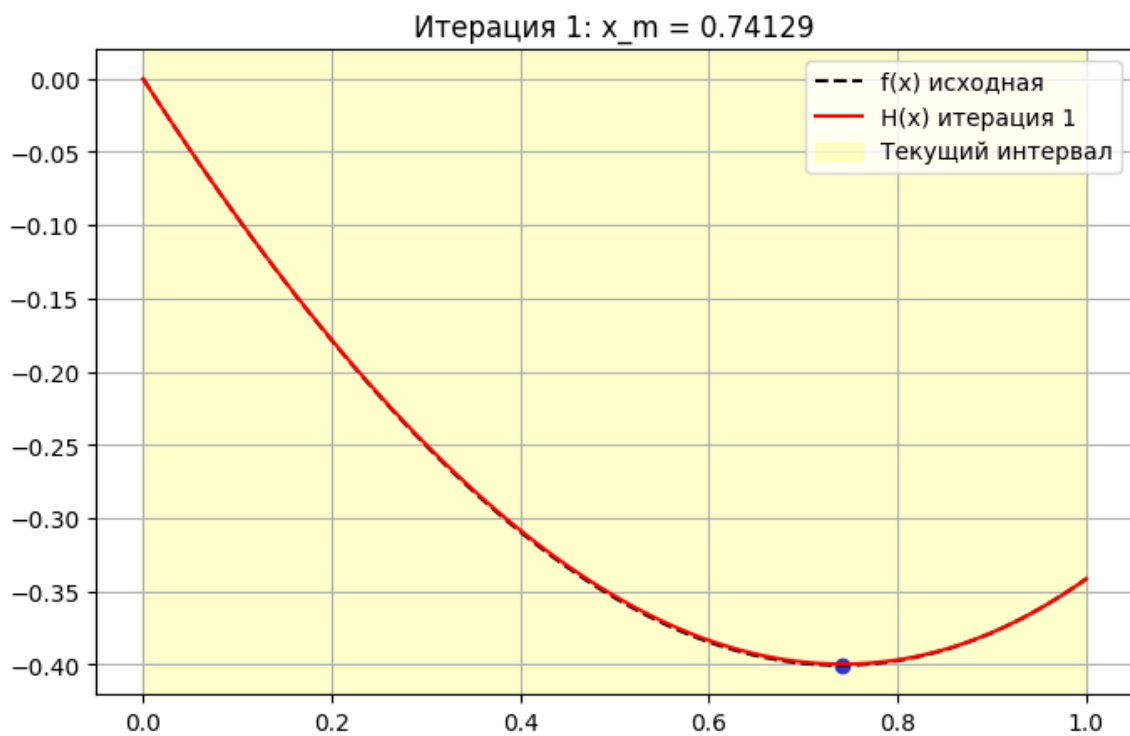


Рис. 1: Итерация 1

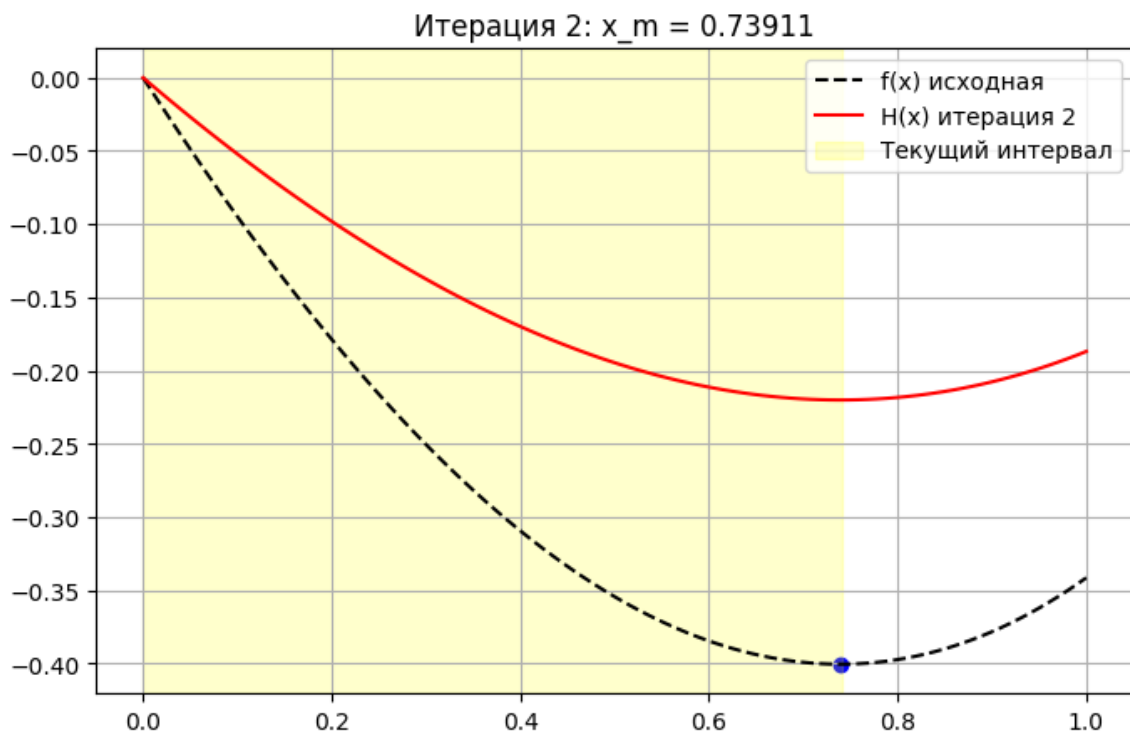


Рис. 2: Итерация 2

4 Анализ решения

В ходе выполнения работы были получены следующие данные:

- **Итерация 1:** Интервал $[0, 1]$. Получено приближение $x_m \approx 0.733$.
- **Итерация 2:** Интервал сузился. x_m уточняется до тех пор, пока производная не станет меньше 10^{-4} .
- **Результат:** Точка минимума найдена за минимальное количество шагов (обычно 2-3 итерации для данной функции), что подтверждает эффективность метода кубической аппроксимации по сравнению с линейными методами.